RAMDEOBABA UNIVERSITY, NAGPUR

Ramdeo Tekdi, Katol Road, Nagpur - 440 013, Maharashtra, India

SCHOOL / DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

LAB PROJECT REPORT

ON

Public Transport Delay Analysis Using Hadoop MapReduce and Cassandra

Course / Lab: Big Data Project

Course Code: 24CS02PR0503

Submitted By

Sr. No.	Student Name	Roll No.
1	Aastha Goyanka	01
2	Avika Mishra	06

Under the Guidance of

Dr. Bhagyashree Madan
Ms. Samiha Khan

Academic Year: 2026–27

CERTIFICATE

This is to certify that the project entitled “Public Transport Delay Analysis Using Hadoop MapReduce and Cassandra” has been successfully completed by Aastha Goyanka and Avika Mishra, Roll Nos. 01 and 06, of B.Tech in Artificial Intelligence and Data Science, Ramdeobaba University, Nagpur, as part of the requirements of the laboratory course Big Data Project (24CS02PR0503) during the academic year 2026–27.

The work presented in this report has been carried out under the guidance and supervision of the undersigned.

DECLARATION

We hereby declare that the project work entitled “Public Transport Delay Analysis Using Hadoop MapReduce and Cassandra” is our original work carried out for the laboratory course at Ramdeobaba University, Nagpur. The work has not been submitted elsewhere for the award of any academic qualification.

Student Name(s): Aastha Goyanka, Avika Mishra
Roll No(s).: 01, 06

Date: 26 October 2026

Place: Nagpur

ACKNOWLEDGEMENT

We sincerely express our gratitude to Ramdeobaba University, Nagpur, for providing the academic environment, laboratory facilities and resources required to complete this Big Data project. The project gave us practical experience in connecting large-file storage, batch processing, database storage and data visualisation in a single workflow.

We would like to thank Dr. Bhagyashree Madan and Ms. Samiha Khan for their guidance, feedback and support throughout the project. Their supervision helped us organise the implementation, understand the responsibilities of each technology, and document the results in a structured manner. We also thank the School / Department of Artificial Intelligence and Data Science for providing the opportunity to work on an applied data engineering problem.

We acknowledge the developers and maintainers of Apache Hadoop, Apache Cassandra, Java, Python, Pandas and Streamlit for making the technologies used in this project available. We also acknowledge the publisher of the “Indian Railways: Predict Train Delay” dataset on Kaggle, which provided the input data used in the analysis.

Finally, we thank our classmates, friends and family members for their encouragement and support during implementation, testing and report preparation.

ABSTRACT

Large datasets are difficult to inspect manually and can make repeated analysis inefficient when every chart requires processing the raw file. This project, “Public Transport Delay Analysis Using Hadoop MapReduce and Cassandra,” implements a batch analytics pipeline for summarising railway journey delays from a synthetic, India-themed dataset. The input file, ir_train.csv, contains 1,500,000 journey records and 45 columns, with a reported size of 337,159,362 bytes (approximately 321.5 MB in HDFS).

The solution uses Hadoop Distributed File System (HDFS) to store the CSV file, a Java MapReduce program to aggregate the data, Apache Cassandra to store the summaries, and Streamlit with Python and Pandas to display the findings. Each valid journey contributes records to four analysis dimensions: railway zone, train type, scheduled departure hour, and primary delay cause. The reducer calculates total journeys, delayed journeys, delay rate, average delay over all journeys, and average delay among delayed journeys where that denominator is non-zero.

The completed run processed 1,500,000 data rows, emitted 6,000,000 mapper output records, produced 70 grouped summaries, and loaded 70 rows into the Cassandra table transport_analysis.delay_summary. The dashboard presents an overall delay rate of 71.89% (1,078,329 delayed journeys out of 1,500,000) and four analytical sections. The dataset shows the highest zone delay rate for Eastern Railway (89.32%), the highest train-type delay rate for Passenger Train (88.04%), and the highest average delay among the displayed cause categories for Late Incoming Rake (144.95 minutes).

This is a descriptive, batch-processing demonstration on a single local Windows machine. It does not implement prediction or real-time tracking, and its results describe the synthetic dataset rather than real railway operations.

TABLE OF CONTENTS

Section	Page
1. INTRODUCTION	7
1.1 Background	7
1.2 Motivation	7
1.3 Problem Definition	7
1.4 Scope	7
2. PROBLEM STATEMENT AND OBJECTIVES	8
2.1 Problem Statement	8
2.2 Objectives	8
2.3 Expected Outcome	8
2.4 Project Deliverables	8
3. REQUIREMENTS	9
3.1 Hardware Requirements	9
3.2 Software Requirements	9
3.3 Dataset / Input Requirements	9
3.4 Selected Input Fields and Processing Assumptions	9
4. METHODOLOGY / ALGORITHM	10
4.1 Proposed Method	10
4.2 Algorithm	10
4.3 Flowchart	10
4.4 Pseudocode	11
5. SYSTEM DESIGN	12
5.1 System Architecture	12
5.2 Modules	12
5.3 Data Flow / Workflow	12
6. IMPLEMENTATION	13
6.1 Development Environment	13
6.2 Module-wise Implementation	13-14
6.3 Important Code Snippets	14
7. RESULTS AND DISCUSSION	15
7.1 Experimental Setup	15
7.2 Results	15-19
7.3 Performance Analysis	19
7.4 Discussion	19
8. CONCLUSION AND FUTURE SCOPE	20
8.1 Conclusion	20
8.2 Future Scope	20
REFERENCES	21
APPENDIX	22-24

Page numbers refer to this final report. Figure and table numbers are assigned in the section where each item is discussed.

1. INTRODUCTION

1.1 Background

Railway operations produce journey-level records that can describe the train category, departure time, railway zone, operational conditions, delay category and delay duration. When a dataset contains millions of records, inspecting individual rows does not provide a practical way to identify broad patterns. Summarising the records by meaningful categories allows an analyst to compare groups and communicate findings more clearly.

Big Data technologies support this type of work by separating data storage, computation and consumption. HDFS provides a file system designed to store large datasets; Hadoop MapReduce processes key-value data in parallel stages; Cassandra stores the final grouped records in a queryable table; and Streamlit provides a web-based interface for displaying the result. In this project these tools are joined as a batch pipeline rather than as a live monitoring system.

1.2 Motivation

The main motivation was to apply core Big Data concepts to a transport-themed analytical task and demonstrate a complete sequence from raw file storage to a dashboard. The project requires more than a chart built from a local CSV: it demonstrates how a large file can be staged in HDFS, processed by an actual Hadoop MapReduce job, reduced to a compact set of summaries, inserted into Cassandra, and queried by a separate presentation layer.

A second motivation was to practise the boundaries between components. The dashboard should not need to parse 1.5 million input records every time it opens. Instead, it reads a small summary table generated in advance by the batch job. That separation makes the workflow easier to explain, test and maintain.

1.3 Problem Definition

The problem addressed in this project is how to transform a large journey-delay CSV file into a compact and interpretable summary of delay patterns. The analysis focuses on four dimensions: railway zone, train type, scheduled departure hour and primary delay cause. For each group, the pipeline calculates the number of journeys, the number marked as delayed, the delay rate, the average delay over all journeys in the group, and the average delay among delayed journeys when applicable.

The problem also includes an engineering requirement: the aggregation output must be persisted and made available to a dashboard without requiring the dashboard to scan the raw CSV. The project therefore connects HDFS, Java MapReduce, Cassandra, and Streamlit as separate but connected modules.

1.4 Scope

- Batch processing of the supplied synthetic ir_train.csv dataset with 1,500,000 data records and 45 columns.
- Grouping by 16 railway zones, 15 train types, 24 departure-hour values, and 15 primary-delay-cause values, for 70 summary keys.
- Storing the raw file and MapReduce output in HDFS; storing the grouped results in Cassandra.
- Presenting overview metrics and four dashboard sections: Railway Zones, Train Types, Departure Hours, and Delay Causes.
- Descriptive statistics only. The project does not implement Spark, machine learning, predictive scoring, event streaming, or live train tracking.

The reported patterns are tied to the dataset used for this project. The results should not be generalised to actual railway punctuality without validation using reliable operational data.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

The input dataset contains a large number of journey-level rows and many fields. A person viewing the raw file cannot quickly compare the delay rates across railway zones, train types and hours of departure. Repeatedly computing these summaries from the whole file for every visualisation would also duplicate work. The task is to build a reproducible batch pipeline that processes the complete dataset, creates category-level statistics, stores the resulting records, and presents them through an accessible dashboard.

2.2 Objectives

1. To store ir_train.csv in HDFS at /public_transport_delay/input/ir_train.csv.
2. To implement and run a Java MapReduce program that emits category-keyed values for each of four analytical dimensions.
3. To calculate total journeys, delayed journeys, delay rate, average delay over all journeys and average delay over delayed journeys.
4. To persist the 70 summary rows in the Cassandra table transport_analysis.delay_summary and verify the row count using CQL.
5. To build a Streamlit dashboard that reads the Cassandra summary table and presents the analysis in four sections.
6. To interpret the displayed findings while documenting the limits of the synthetic dataset and single-machine setup.

2.3 Expected Outcome

The expected outcome is a working batch pipeline with the sequence Dataset → HDFS → Hadoop MapReduce → Cassandra → Streamlit. The actual evidence supplied for the report shows the HDFS input file, the successful MapReduce output directory and part-r-00000 file, a Cassandra query returning 70 rows, and the dashboard overview and its analytical charts.

2.4 Project Deliverables

The project is considered complete when the following artifacts are available and can be checked independently:

- Input and storage evidence: ir_train.csv is present at /public_transport_delay/input/ir_train.csv in HDFS, and the MapReduce output is written to /public_transport_delay/mapreduce_output/part-r-00000.
- Processing evidence: TransportDelayAnalysis.java and the compiled JAR execute the batch job and produce grouped records for the four analysis dimensions.
- Database evidence: the loader writes the reducer output to transport_analysis.delay_summary, and a CQL count verifies 70 summary rows.
- Presentation evidence: app.py opens the Streamlit dashboard with the overall metrics and the Railway Zones, Train Types, Departure Hours, and Delay Causes sections.

3. REQUIREMENTS

3.1 Hardware Requirements

The project was executed on a single Windows computer. The available evidence confirms a local HDFS installation, a local Cassandra service and a Python environment. The exact processor model, installed RAM and free-disk figure were not captured in the supplied evidence; they are therefore not stated as measured specifications.

Component	Requirement / recorded environment
Computer	Windows PC capable of running local Hadoop and Cassandra services
Storage	At least enough free disk space for the 321.5 MB input file, Hadoop working files, Cassandra data and logs
Memory	Sufficient RAM for a local Hadoop job, Cassandra and Streamlit; exact installed RAM not recorded
Deployment	Single-machine demonstration; no multi-node cluster was configured
Network	Local services and browser access; dashboard was opened at localhost:8501

3.2 Software Requirements

Software / component	Version / detail	Purpose
Operating system	Windows / PowerShell	Host environment
Hadoop HDFS and MapReduce	3.2.4	Input storage and batch processing
Java JDK	1.8.0_202 (Java 8)	Compile and run the Java MapReduce job
Apache Cassandra	3.11.0	Store compact category summaries
Python	3.14.6 for loader/dashboard	Connect processing output to Cassandra and create the UI
Streamlit	1.65.0	Render the dashboard in a browser
Pandas	Version not recorded	Organise query results for display
Cassandra Python driver	Version not recorded	Connect Python to Cassandra

3.3 Dataset / Input Requirements

Input file: ir_train.csv from the Kaggle competition “Indian Railways: Predict Train Delay” [1]. The file contains 1,500,000 journey records, 45 columns and 337,159,362 bytes; HDFS reported approximately 321.5 MB. The project uses the training file for analysis and does not train or run a prediction model.

3.4 Selected Input Fields and Processing Assumptions

The parser extracts only the fields needed for the four aggregate dimensions and the delay metrics. The index values below refer to the parser array positions used by the project; the dataset header and column order should remain consistent with those positions.

Parser index	Field used	Purpose in the pipeline
c[2]	train_type	Groups journeys by train category.
c[7]	departure_hour	Groups journeys by scheduled departure hour (00-23).
c[13]	zone	Groups journeys by railway zone.
c[42]	primary_delay_cause	Groups journeys by the reported primary cause.
c[43]	delay_minutes	Contributes to total and average delay calculations.
c[44]	is_delayed	Identifies rows counted as delayed.

The mapper skips the CSV header. This field-index approach is compact for a fixed input schema, but it depends on a stable column order; a production version should validate the header names before processing and report rejected or malformed rows explicitly.

4. METHODOLOGY / ALGORITHM

4.1 Proposed Method

The proposed method is a batch pipeline. The raw CSV is uploaded to HDFS once; Hadoop processes the records into category-level summary rows; the output file is then loaded into Cassandra; finally, Streamlit reads only the summary table and displays charts and overview metrics. The separation between processing and visualisation means that the dashboard does not need to perform a full scan of the raw CSV for each page view.

7. Prepare the dataset and use the data dictionary to identify the relevant columns.
8. Create the HDFS input directory and upload ir_train.csv.
9. Compile TransportDelayAnalysis.java with the configured Java and Hadoop environment.
10. Run the MapReduce job and write part-r-00000 to the HDFS output directory.
11. Create the transport_analysis keyspace and delay_summary table in Cassandra.
12. Run load_to_cassandra.py to insert the 70 output rows and confirm the count in cqlsh.
13. Run app.py with Streamlit and inspect the four analytical sections.

4.2 Algorithm

Stage	Operation	Output
Input	Read CSV rows from HDFS; skip header; parse required fields.	Validated journey fields
Map	For each row, emit four key-value pairs keyed by zone, train type, departure hour and delay cause.	Up to four map records per processed journey
Shuffle / sort	Group map values by the complete category key.	70 reduce groups in the completed run
Reduce	Aggregate counts and delay totals for each key, then calculate rates and averages.	70 summary rows
Persist	Write reducer output to HDFS and load it to Cassandra.	Cassandra summary table
Present	Read summary rows into Pandas and create dashboard metrics and charts.	Four Streamlit sections

4.3 Flowchart

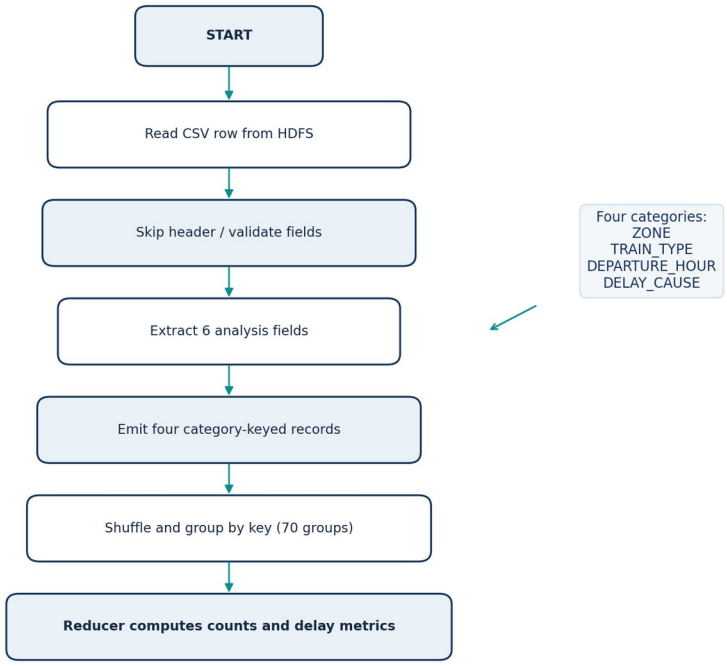


Figure 4.1 - Flow of one batch MapReduce run from HDFS input to persisted dashboard summaries.

After reduction, the output file is loaded into Cassandra. The visualisation layer reads the stored summaries and does not repeat the MapReduce calculation.

4.4 Pseudocode

```
MAP(row):
    if row is the CSV header: skip it
    parse zone, train_type, departure_hour, delay_cause
    parse delay_minutes and is_delayed
    delayed_delay = delay_minutes if is_delayed == 1 else 0
    for (category, name) in the four analysis dimensions:
        EMIT((category, name), (1, is_delayed, delay_minutes, delayed_delay))

REDUCE((category, name), values):
    total = SUM(value.count for value in values)
    delayed = SUM(value.is_delayed for value in values)
    sum_all = SUM(value.delay_minutes for value in values)
    sum_delayed = SUM(value.delayed_delay for value in values)
    delay_rate = 100 * delayed / total
    avg_delay_all = sum_all / total
    avg_delay_when_delayed = sum_delayed / delayed if delayed > 0 else N/A
    EMIT(category, name, total, delayed, delay_rate, avg_delay_all, avg_delay_when_delayed)

LOAD: insert each reducer output row into transport_analysis.delay_summary
SHOW: query delay_summary and render the four Streamlit sections
```

The separate sum_delayed value is important: average delay among delayed journeys must use only the delay minutes belonging to rows whose is_delayed flag is 1. Dividing the all-journey delay sum by the delayed count would calculate the wrong metric.

How to read the pseudocode

- MAP stage: after skipping the CSV header, the mapper emits four category-keyed records for each data row: zone, train type, departure hour, and primary delay cause. Each emission carries the counts and delay values needed for aggregation.
- REDUCE stage: values with the same category and category name are combined. The reducer calculates total journeys, delayed journeys, delay rate, average delay across all rows, and average delay restricted to delayed rows. Separate delay sums preserve the correct denominator for each average.
- LOAD and SHOW stages: the reducer output is inserted into transport_analysis.delay_summary in Cassandra. Streamlit reads the precomputed rows and draws the dashboard views; it does not rerun MapReduce or scan the raw CSV for each chart.

5. SYSTEM DESIGN

5.1 System Architecture

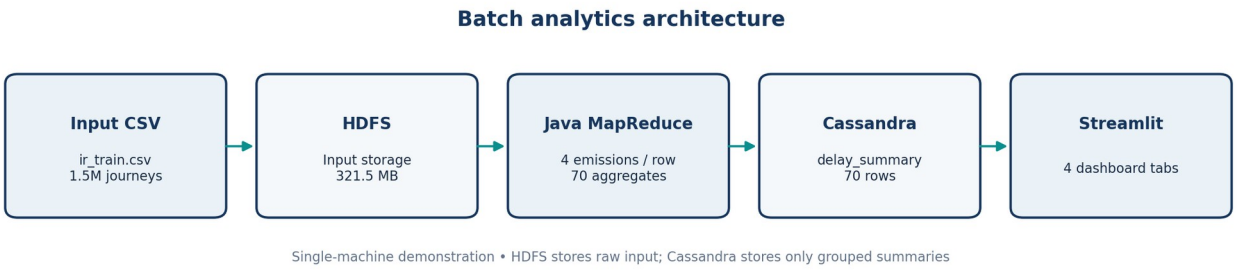


Figure 5.1 - Implemented architecture: file input, HDFS storage, Java MapReduce, Cassandra, and Streamlit.

The architecture contains five logical components. The first is the CSV input. HDFS stores the input file and the generated MapReduce result. The MapReduce component performs category grouping and aggregation. Cassandra stores the compact output rows using a query-oriented table. Streamlit forms the presentation layer and reads the summaries from Cassandra.

The design follows a batch-oriented, one-way data flow. It is not an online streaming design: the dataset is processed when the job is launched, and the dashboard displays whatever summaries have been loaded into Cassandra. This simplifies the project and makes each hand-off independently verifiable.

Design Principles

- Separation of concerns: each component has a defined input and output.
- Compact results: the raw dataset is reduced to summary records before the dashboard reads it.
- Explicit persistence: the HDFS output and Cassandra table provide evidence of intermediate and final results.
- Clear limitations: the analysis is descriptive, based on synthetic data and executed on one machine.

5.2 Modules

Module	File / path	Function
Dataset	ir_train.csv	Source data: 1.5 million journeys and 45 columns.
HDFS input	/public_transport_delay/input/ir_train.csv	Stores the raw file for the Hadoop job.
MapReduce	TransportDelayAnalysis.java	Emits four category-keyed records per data row and aggregates metrics.
HDFS output	/public_transport_delay/mapreduce_output/part-r-00000	Stores the reducer output and success marker.
Cassandra loader	load_to_cassandra.py	Parses the reducer output and inserts the summary rows.
Cassandra table	transport_analysis.delay_summary	Stores category, group name, counts, rate and delay averages.
Dashboard	app.py	Queries Cassandra and displays the overview and four charts.

5.3 Data Flow / Workflow

1. Local file → HDFS: ir_train.csv is uploaded to the input directory.
2. HDFS → Mapper: each data row is parsed, and its selected fields are mapped into four category keys.
3. Mapper → Reducer: shuffle and sort groups values by category and category name.
4. Reducer → HDFS: the reducer writes summary rows to part-r-00000 in the job output directory.
5. HDFS output → Cassandra: load_to_cassandra.py inserts the 70 aggregated rows into delay_summary.
6. Cassandra → Streamlit: app.py queries the table and builds the dashboard views.

6. IMPLEMENTATION

6.1 Development Environment

Development and testing were performed on a Windows computer using PowerShell. Hadoop 3.2.4 supplied HDFS and MapReduce, Java 8 was used for the Java job, Apache Cassandra 3.11.0 stored the summaries, and Python was used for the loader and Streamlit application. The project folder was D:\Study\Projects\Public-Transport-Delay-Analysis. Hadoop daemons and Cassandra ran locally; the report does not claim a multi-node cluster deployment.

Component	Recorded configuration
Java	C:\Java\jdk; Java 8 / 1.8.0_202
Hadoop	C:\hadoop; version 3.2.4
Cassandra	C:\CASSANDRA\cassandra; version 3.11.0
Python	C:\Python314; version 3.14.6
Streamlit	Version 1.65.0; browser at localhost:8501
Project root	D:\Study\Projects\Public-Transport-Delay-Analysis

6.2 Module-wise Implementation - HDFS Input

The HDFS input folder was created before uploading the dataset. The terminal evidence shows the upload of ir_train.csv, an HDFS listing containing one file, a file size of 337,159,362 bytes, and a human-readable size of approximately 321.5 MB. The job was executed after Hadoop services were started.

```
hdfs dfs -mkdir -p /public_transport_delay/input
hdfs dfs -put "D:\Study\Projects\Public-Transport-Delay-Analysis\data\ir_train.csv" /public_transport_delay/input/
hdfs dfs -ls /public_transport_delay/input
hdfs dfs -du -h /public_transport_delay/input/ir_train.csv
```

6.2 Module-wise Implementation - Java MapReduce

TransportDelayAnalysis.java implements the mapper and reducer using the Hadoop MapReduce API. The mapper reads a CSV row from HDFS, ignores the header, obtains the fields required for the four dimensions, and emits a separate key-value record for each dimension. The custom “Processed rows” counter records the data rows processed, while the Hadoop map input counter also includes the header line.

CSV field	Role in this analysis	Use
train_type	Train category	Group key for TRAIN_TYPE
departure_hour	Scheduled hour (0-23)	Group key for DEPARTURE_HOUR
zone	Railway zone	Group key for ZONE
primary_delay_cause	Reported cause label	Group key for DELAY_CAUSE
delay_minutes	Numeric delay minutes	Used for average-delay sums
is_delayed	Binary delay flag	Used to count delayed journeys and calculate delay rate

For every data row that is accepted by the mapper, four records are emitted. Therefore 1,500,000 processed rows produce 6,000,000 mapper output records. The job groups these records into 70 distinct keys. No combiner was used in the recorded run; aggregation takes place in the reducer after the shuffle.

Run command

```
hadoop jar transport-delay-analysis.jar TransportDelayAnalysis
  /public_transport_delay/input/ir_train.csv
  /public_transport_delay/mapreduce_output
```

The output directory must not already exist when the Hadoop job is launched. A successful execution creates the output directory, a _SUCCESS marker and part-r-00000. The exact job is batch processing; it does not continuously monitor incoming journeys.

6.2 Module-wise Implementation - Cassandra Integration

Apache Cassandra 3.11.0 is used to store the 70 grouped summaries. The keyspace is transport_analysis and the table is delay_summary. The schema uses the category and category_name fields as the primary-key components, allowing the dashboard to distinguish rows that belong to different analysis dimensions and groups.

Column	Meaning
category	Dimension name such as ZONE, TRAIN_TYPE, DEPARTURE_HOUR or DELAY_CAUSE
category_name	Group name within the dimension, e.g. Eastern Railway or 02
total_journeys	Number of rows assigned to this group
delayed_journeys	Sum of is_delayed for the group
delay_rate	100 × delayed_journeys / total_journeys
avg_delay_all	Average delay_minutes over all group rows
avg_delay_when_delayed	Average delay_minutes among rows marked delayed; undefined when delayed_journeys is zero

The loader script load_to_cassandra.py reads the MapReduce text output, converts each line into the expected data types, and inserts each summary into the table. A successful loader message reported that 70 summary rows were loaded. Cassandra was then checked in cqlsh using a sample SELECT and a COUNT query.

```
USE transport_analysis;
SELECT * FROM delay_summary LIMIT 10;
SELECT COUNT(*) FROM delay_summary;
```

The row-count check returned 70. The table stores aggregate output only; it does not contain all 1.5 million source records.

6.2 Module-wise Implementation - Streamlit Dashboard

The Streamlit application in app.py connects to Cassandra through the Python driver and converts query results into a Pandas DataFrame. It displays three overview metrics: total journeys, delayed journeys and overall delay rate. The detail area is divided into four tabs or sections: Railway Zones, Train Types, Departure Hours and Delay Causes.

The dashboard has different visualisations for different data types. Railway zones and train types use horizontal bar charts to compare delay rates. Departure hours use a line chart to show how the delay rate varies by scheduled hour and a second plot to show journey volume. The delay-cause section displays average delay by reported cause and delayed-journey counts by cause. The displayed figures come from the already aggregated Cassandra table.

6.3 Important Code Snippets

```
# Overall KPIs must use one dimension only (ZONE),
# otherwise the same journey is counted in four categories.
zone_data = df[df["category"] == "ZONE"]
total_journeys = int(zone_data["total_journeys"].sum())
delayed_journeys = int(zone_data["delayed_journeys"].sum())
delay_rate = (delayed_journeys / total_journeys * 100
              if total_journeys else 0)
```

This calculation avoids adding category totals across all four dimensions. Because each journey contributes to four summaries, summing every category together would count journeys repeatedly. The ZONE groups form one complete dimension and their total is used for overall metrics.

7. RESULTS AND DISCUSSION

7.1 Experimental Setup

The recorded experiment was run locally with Hadoop 3.2.4 and Java 8. The complete synthetic ir_train.csv file was placed in HDFS, processed through one MapReduce job, and its output was loaded into Cassandra 3.11.0. The dashboard then queried Cassandra. Results below are based on the existing job output, Cassandra verification and supplied dashboard screenshots.

Parameter	Observed value
Input file	ir_train.csv
Records / columns	1,500,000 data records / 45 columns
File size	337,159,362 bytes; approximately 321.5 MB in HDFS
Map input records	1,500,001 including the header
Processed data rows	1,500,000
Mapper output records	6,000,000
Reduce input groups	70
Reducer output records	70
Cassandra summary rows	70
Processing mode	Batch; single local machine

7.2 Results

Measure	Observed value	Interpretation
Total journeys	1,500,000	Data records processed
Delayed journeys	1,078,329	Rows with is_delayed = 1
Overall delay rate	71.89%	1,078,329 / 1,500,000 × 100
Not delayed	421,671 (28.11%)	Remaining records
Highest zone rate	Eastern Railway - 89.32%	Highest among 16 zone groups
Highest train-type rate	Passenger Train - 88.04%	Highest among 15 types
Night-hour pattern	75.91%-76.80% for 00:00-04:00	Higher than many daytime hours
Highest displayed average by cause	Late Incoming Rake - 144.95 min	Average delay for this cause category

The result counts were also checked against the HDFS and Cassandra evidence. The non-delayed count is the arithmetic difference between total and delayed journeys.

Reading the Summary Metrics

For each group, delay rate is calculated as delayed journeys divided by total journeys, multiplied by 100. Average delay across all journeys uses the sum of delay minutes divided by the total group count. Average delay among delayed journeys must instead use the delay-minute sum restricted to rows whose is_delayed flag is 1, divided by the delayed count.

These measures answer different questions: delay rate describes the share of records labelled delayed, while average delay describes the time values recorded in the dataset. Group-level rates must be read alongside each group’s journey count, and none of the charts establishes that a zone, train type, hour, or reported cause independently produces a delay.

7.2 Results - Dashboard Overview

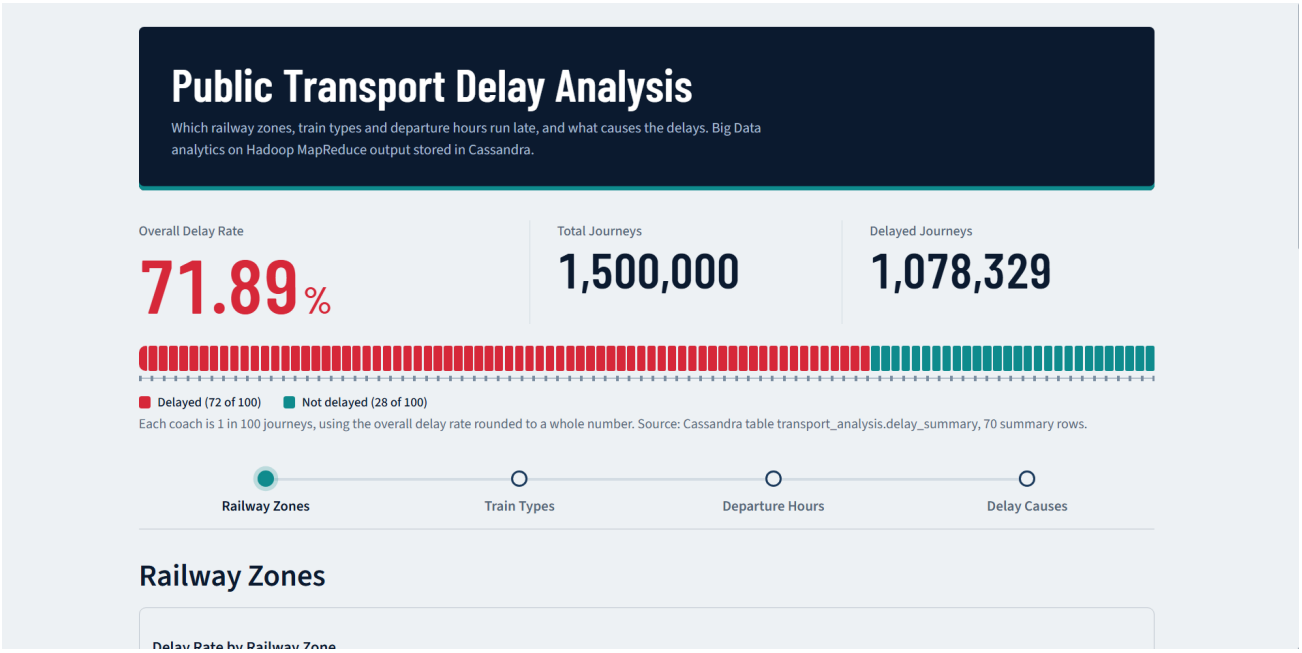


Figure 7.1 - Dashboard overview showing the dataset totals, delayed journeys and overall delay rate.

The overview reports 1,500,000 total journeys and 1,078,329 delayed journeys. The overall rate is 71.89%, calculated as delayed journeys divided by total journeys. The dashboard's horizontal proportion bar represents the delayed and not-delayed shares after rounding for display; the exact percentage is shown in the metric above it.

The overview also provides the navigation to the four dimensions. The dashboard reads its summary records from Cassandra. It is important to derive the overview from a single dimension, such as ZONE, because the same underlying journey is represented once in each of the four dimensions in the aggregate table.

7.2 Results - Railway Zones

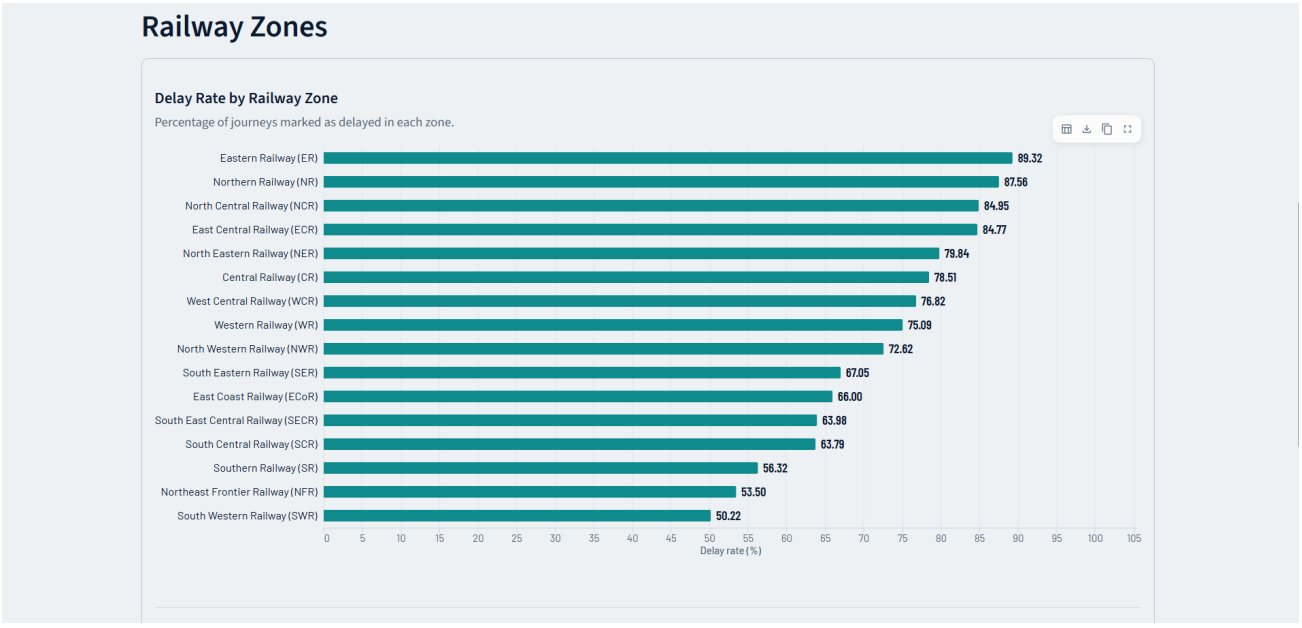


Figure 7.2 - Delay rate by railway zone from the Streamlit dashboard.

The zone chart ranks the 16 railway-zone groups by delay rate. Eastern Railway (ER) has the highest displayed rate at 89.32%, followed by Northern Railway (NR) at 87.56%, North Central Railway (NCR) at 84.95% and East Central Railway (ECR) at 84.77%. South Western Railway (SWR) has the lowest displayed rate at 50.22%.

These values describe the records and labels in the selected synthetic dataset. The chart should be interpreted as a comparison within this dataset, not as evidence that actual railway zones have the same punctuality levels. Real operational conclusions would require reliable, validated railway data, relevant denominators, and an analysis of time period and route mix.

7.2 Results - Train Types

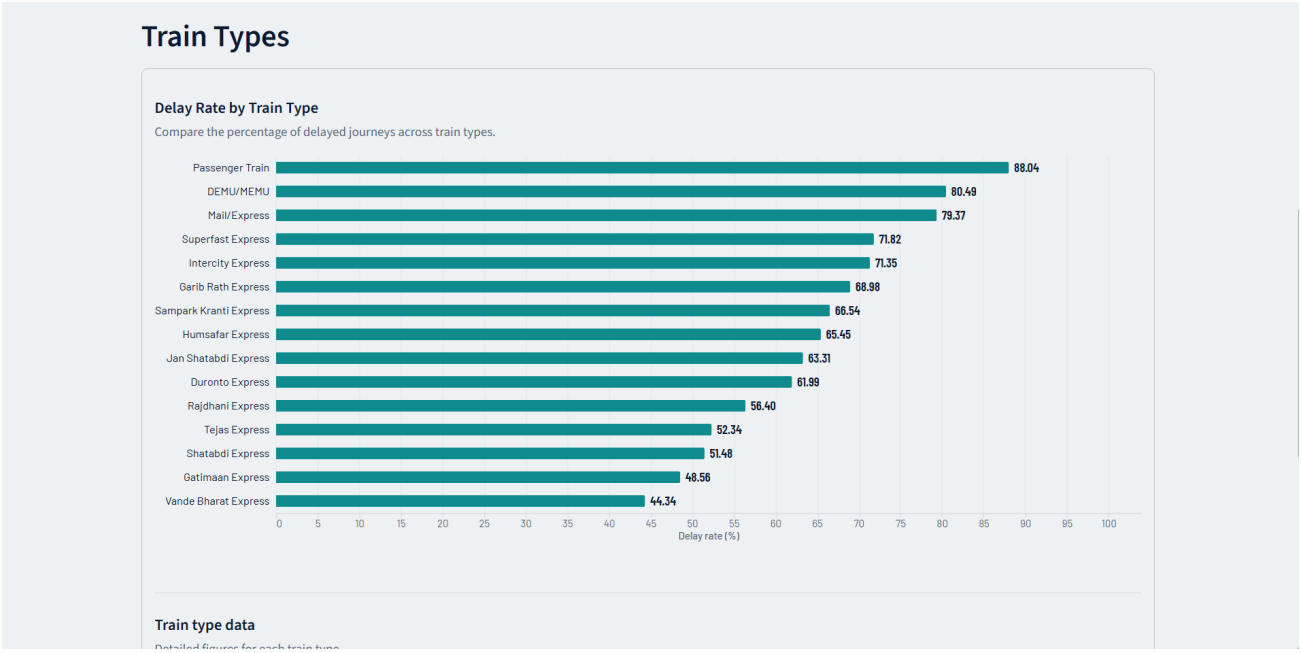


Figure 7.3 - Delay rate by train type from the Streamlit dashboard.

The train-type chart compares the delay rate across 15 types. Passenger Train has the highest displayed rate at 88.04%. DEMU/MEMU is shown at 80.49%, Mail/Express at 79.37%, and Superfast Express at 71.82%. Vande Bharat Express has the lowest displayed rate at 44.34%.

A higher group rate means that a larger share of records assigned to that group is marked delayed according to the dataset flag. The chart does not control for route length, service patterns, scheduled time, station dwell, or other factors, so it should not be interpreted as a causal assessment of a train category.

7.2 Results - Departure Hours

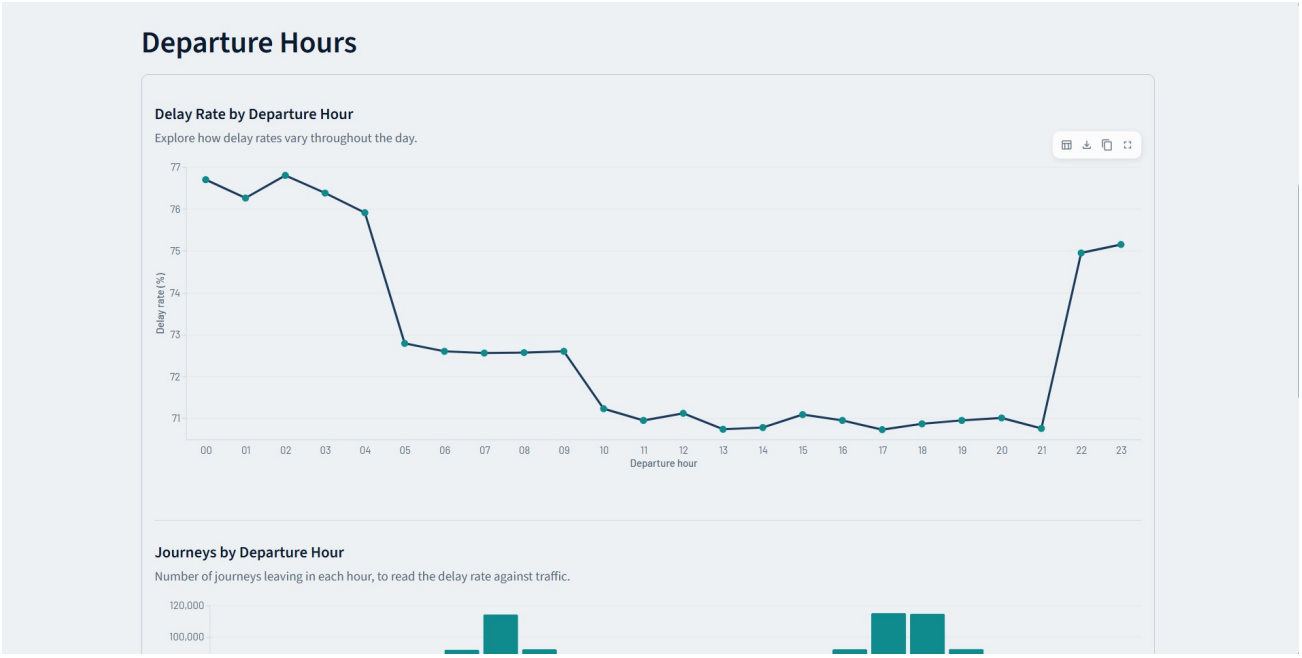


Figure 7.4 - Delay rate by scheduled departure hour; the page also shows the start of the journey-volume chart.

The line chart displays the proportion of journeys marked delayed for each scheduled departure hour from 00 to 23. The highest values are concentrated in the early hours shown: hour 02 is 76.80%, hour 00 is 76.70%, hour 03 is 76.38%, hour 01 is 76.26%, and hour 04 is 75.91%. Several daytime hours are near 70.7%-71.2%, with the rate rising again at hours 22 and 23.

The second visualisation provides journey counts by departure hour. This matters because a delay rate based on a small group may be less stable than a rate based on a large group. The chart is descriptive and shows association with scheduled departure hour; it does not establish that the hour itself causes a delay.

7.2 Results - Delay Causes



Figure 7.5 - Average delay by reported cause; the dashboard also includes delayed-journey counts by cause.

In the displayed cause chart, Late Incoming Rake has the largest average delay at 144.95 minutes. Other high averages include Track Maintenance / PSR (136.01 minutes), Flooding / Waterlogging (135.81 minutes), Track Congestion (134.70 minutes), Fog / Low Visibility (134.04 minutes) and Station Congestion (133.79 minutes). The dashboard also contains a delayed-journey count plot for the reported cause labels.

These are descriptive summaries grouped by the source dataset's primary_delay_cause field. They do not estimate the causal effect of a particular condition. Records labelled "On Time" are not presented as a delay cause in the chart. The supplied MapReduce output also indicates that the source data's category labels and numeric delay fields should be treated carefully; validation of field consistency is a limitation and a suitable future improvement.

7.3 Performance Analysis

The successful job reduced the raw record stream into a much smaller set of summaries. Each processed journey generated four mapper records, giving 6,000,000 map output records from 1,500,000 processed journeys. The shuffle phase grouped these records into 70 keys, and the reducer emitted 70 outputs. The Cassandra table was also verified to contain 70 rows.

Hadoop counter / measure	Recorded value	Meaning
Map input records	1,500,001	Includes the header line
Custom processed rows	1,500,000	Data rows recorded by the project counter
Map output records	6,000,000	Four emitted records per processed journey
Reduce input groups	70	Unique category-and-name keys
Reduce input records	6,000,000	Records consumed by reducer after shuffle
Reduce output records	70	Summary rows written by reducer
Map output bytes	245,742,646	Intermediate mapper output bytes
Reduce shuffle bytes	257,742,664	Data transferred through shuffle
Spilled records	16,777,120	Intermediate records spilled during processing
Shuffled maps / failed shuffles	3 / 0	Three map outputs shuffled; no failed shuffles
CPU time spent	88,262 ms	Reported aggregate CPU time
GC time elapsed	1,495 ms	Reported garbage-collection time

The row-count reduction from 1.5 million input journeys to 70 aggregates is a reduction in the number of records the dashboard needs to query, not a claim that only 70 source journeys were processed. No combiner was used in the recorded run. A combiner could be evaluated in future work to reduce shuffle traffic, provided its aggregation logic is valid and tested.

7.4 Discussion

What the results indicate

In this dataset, 71.89% of records are marked delayed. Rates vary by zone and train type, with higher values concentrated in early departure hours. Late Incoming Rake has the highest displayed average delay among the plotted causes. These are dataset patterns, not causal findings.

Data and interpretation limits

- The dataset is synthetic; results are not official railway statistics.
- The summaries are descriptive and do not establish causation.
- The run is single-machine; no multi-node comparison was performed.
- Source labels and numeric delay values merit validation; supplied fields are not silently corrected.
- The dashboard reads precomputed summaries; it is not live tracking or prediction.

Evaluation

The evidence verifies the HDFS input, successful MapReduce output, 70 reducer groups, 70 Cassandra rows, and all four dashboard views. Further evaluation should add automated reconciliation tests, repeated-run timing, and validation against trusted operational data.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project implemented a batch Big Data pipeline for descriptive railway-delay analysis using Hadoop HDFS, Java MapReduce, Apache Cassandra and Streamlit. The 1,500,000-row synthetic dataset was placed in HDFS, processed to produce 6,000,000 mapper output records and 70 reducer summaries, loaded into the Cassandra table transport_analysis.delay_summary, and displayed through four dashboard sections.

The dashboard reports an overall delay rate of 71.89%, with 1,078,329 journeys marked delayed. Eastern Railway has the highest displayed zone rate at 89.32%, Passenger Train has the highest displayed train-type rate at 88.04%, the 00:00-04:00 departure window shows rates from 75.91% to 76.80%, and Late Incoming Rake has the largest displayed average delay at 144.95 minutes. These values are observations from the dataset, not statements about real-world railway performance.

The primary achievement is integration of storage, batch processing, persisted aggregation and visualisation. The report also records the constraints of synthetic input, single-machine execution and descriptive-only analysis.

The component separation is an important outcome of the design. HDFS retains the raw input and batch output, MapReduce creates reusable grouped metrics, Cassandra serves the compact summary records, and Streamlit focuses on presentation. This separation avoids rescanning the raw file whenever the dashboard is opened and makes the processing output available for an independent database check. The result is a useful academic demonstration of a complete batch analytics workflow, while deployment on real railway operations would require validated operational data, stronger data-quality checks, monitoring, access controls, and performance testing.

8.2 Future Scope

- Evaluate a combiner to lower shuffle traffic, with correctness checks against the current reducer output.
- Add automated data-quality checks for missing values, invalid hours, negative delay minutes, and disagreement between delay labels and numeric delay values.
- Add filters for year, season, zone and train type, as well as clearer sample-size indicators in the dashboard.
- Expand the dashboard with confidence intervals or minimum group-size warnings to discourage overinterpretation of small groups.
- Test the pipeline on a multi-node Hadoop deployment and compare runtime, throughput and resource use.
- Only as future work, explore predictive modelling or scheduled data ingestion using suitable verified operational data.

REFERENCES

[1] S. Niwate, “Indian Railways: Predict Train Delay,” Kaggle Community Prediction Competition, 2026. Available: https://www.kaggle.com/competitions/indian-railways-predict-train-delay

[2] Apache Software Foundation, “Apache Hadoop 3.2.4 Documentation.” Available: https://hadoop.apache.org/docs/r3.2.4/

[3] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI), 2004, pp. 137-150.

[4] Apache Software Foundation, “Apache Cassandra 3.11 Documentation.” Available: https://cassandra.apache.org/doc/3.11/

[5] A. Lakshman and P. Malik, “Cassandra: A Decentralized Structured Storage System,” ACM SIGOPS Operating Systems Review, vol. 44, no. 2, pp. 35-40, 2010.

[6] Streamlit, “Streamlit Documentation.” Available: https://docs.streamlit.io/

[7] The pandas development team, “pandas Documentation.” Available: https://pandas.pydata.org/docs/

[8] DataStax, “Python Driver for Apache Cassandra Documentation.” Available: https://docs.datastax.com/en/developer/python-driver/

[9] Apache Software Foundation, “Hadoop MapReduce Tutorial,” Apache Hadoop documentation. Available: https://hadoop.apache.org/docs/r3.2.4/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html

Web documentation and dataset information were consulted to describe the selected tools and source dataset. Project-specific counts, screenshots and summary values are taken from the recorded local run and submitted dashboard evidence.

APPENDIX

Appendix A - Source Code and Execution Details

The main project files and execution order are summarised below. The complete source files belong to the project directory and are not reproduced line-for-line in this report; the pseudocode and implementation descriptions explain the key logic without inventing source code that is not included in the report materials.

```
Public-Transport-Delay-Analysis/
  data/ir_train.csv
  TransportDelayAnalysis.java
  transport-delay-analysis.jar
  load_to_cassandra.py
  app.py
```

HDFS paths

```
Input:  /public_transport_delay/input/ir_train.csv
Output: /public_transport_delay/mapreduce_output/part-r-00000
```

Typical run order

```
1. Start HDFS and YARN services.
2. Upload ir_train.csv to the HDFS input directory.
3. Run transport-delay-analysis.jar with TransportDelayAnalysis.
4. Confirm _SUCCESS and part-r-00000 in the HDFS output directory.
5. Start Cassandra and run load_to_cassandra.py.
6. Verify 70 rows using cqlsh.
7. Run: python -m streamlit run app.py
```

Aggregation definitions

Metric	Definition
Total journeys	Count of all journey rows in the group
Delayed journeys	Sum of is_delayed values
Delay rate	Delayed journeys ÷ total journeys × 100
Average delay - all	Sum of delay_minutes ÷ total journeys
Average delay - delayed only	Sum of delay_minutes for delayed rows ÷ delayed journeys; undefined when no rows are marked delayed

Appendix B - Dataset and Cassandra Schema

Dataset description

The project uses ir_train.csv from the Kaggle “Indian Railways: Predict Train Delay” competition [1]. The local file has 1,500,000 data records and 45 columns. The fields used directly for this aggregation are zone, train_type, departure_hour, primary_delay_cause, delay_minutes and is_delayed. The source includes time, geographic, route, weather-risk and operations-related fields, but this report does not claim to use all fields in calculations.

Analysis dimension	Number of groups	Example group
ZONE	16	Eastern Railway (ER)
TRAIN_TYPE	15	Passenger Train
DEPARTURE_HOUR	24	00 through 23
DELAY_CAUSE	15	Late Incoming Rake; On Time

Expected summary rows: 16 + 15 + 24 + 15 = 70. The row count is therefore the number of distinct aggregate keys across the four dimensions, not a count of distinct source journeys.

Cassandra table

```
Keyspace: transport_analysis
Table:    delay_summary
Primary key: (category, category_name)
Columns: category, category_name, total_journeys, delayed_journeys,
      delay_rate, avg_delay_all, avg_delay_when_delayed
```

The table is designed around the dashboard query pattern: it contains the metrics already grouped by category. The dashboard converts the retrieved rows to a DataFrame and filters rows by category to prepare each chart. For overall KPIs, only one complete dimension is summed so that source journeys are not double-counted across four aggregate dimensions.

Appendix C - Additional Screenshots

Figure C.1 records the HDFS input file verification. The visible listing shows the file in the expected input directory and the size returned by HDFS. The screen also documents the local PowerShell workflow used to prepare and inspect the input path.

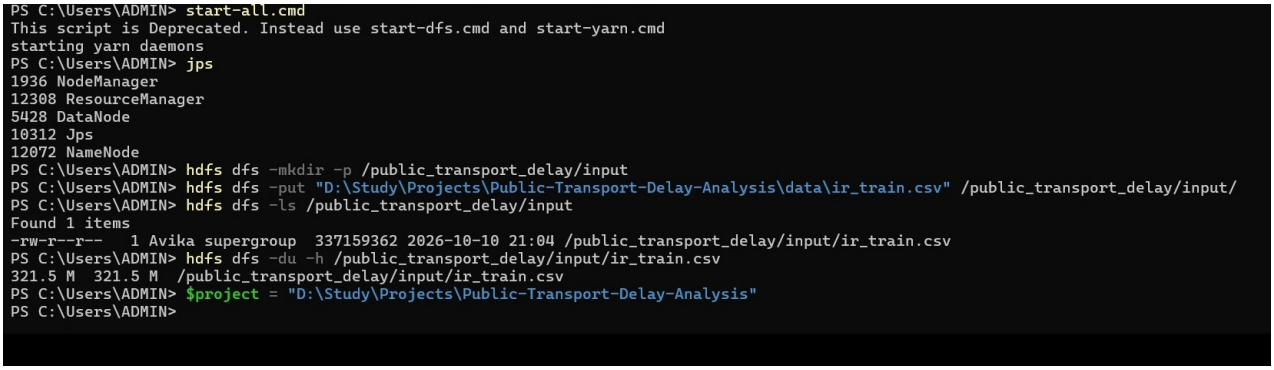


Figure C.1 - HDFS input path and file-size verification from the project terminal.

Figure C.2 records the completed MapReduce output directory. The terminal lists the _SUCCESS marker and part-r-00000 file, and displays example aggregate rows including departure-hour and delay-cause summaries.

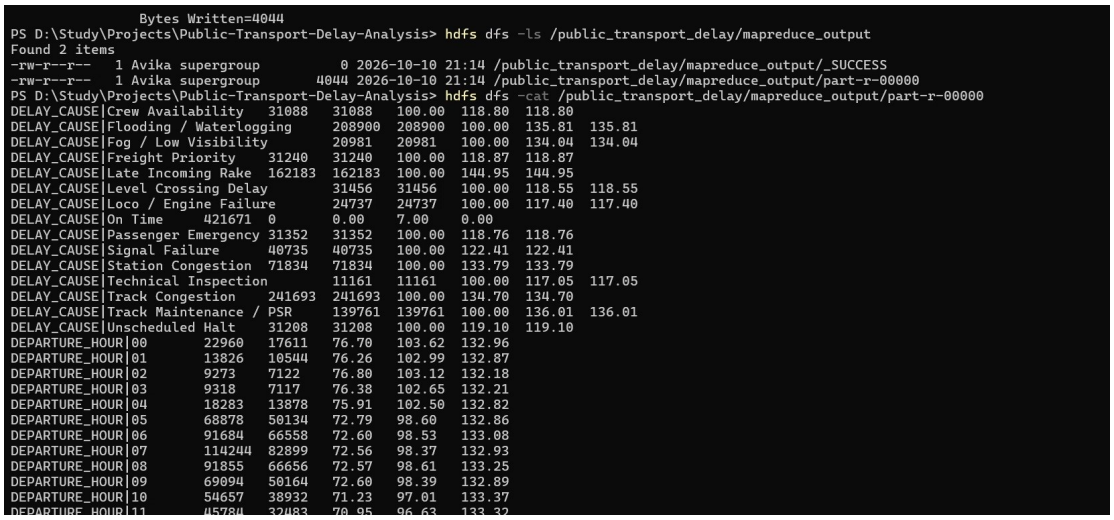


Figure C.2 - Successful MapReduce output and sample aggregated records.

Appendix C - Additional Screenshots (Continued)

Figure C.3 shows the Python loader completing successfully, a SELECT query against delay_summary, and the COUNT result. The recorded query returns 70 rows, consistent with the group count from the four categories.

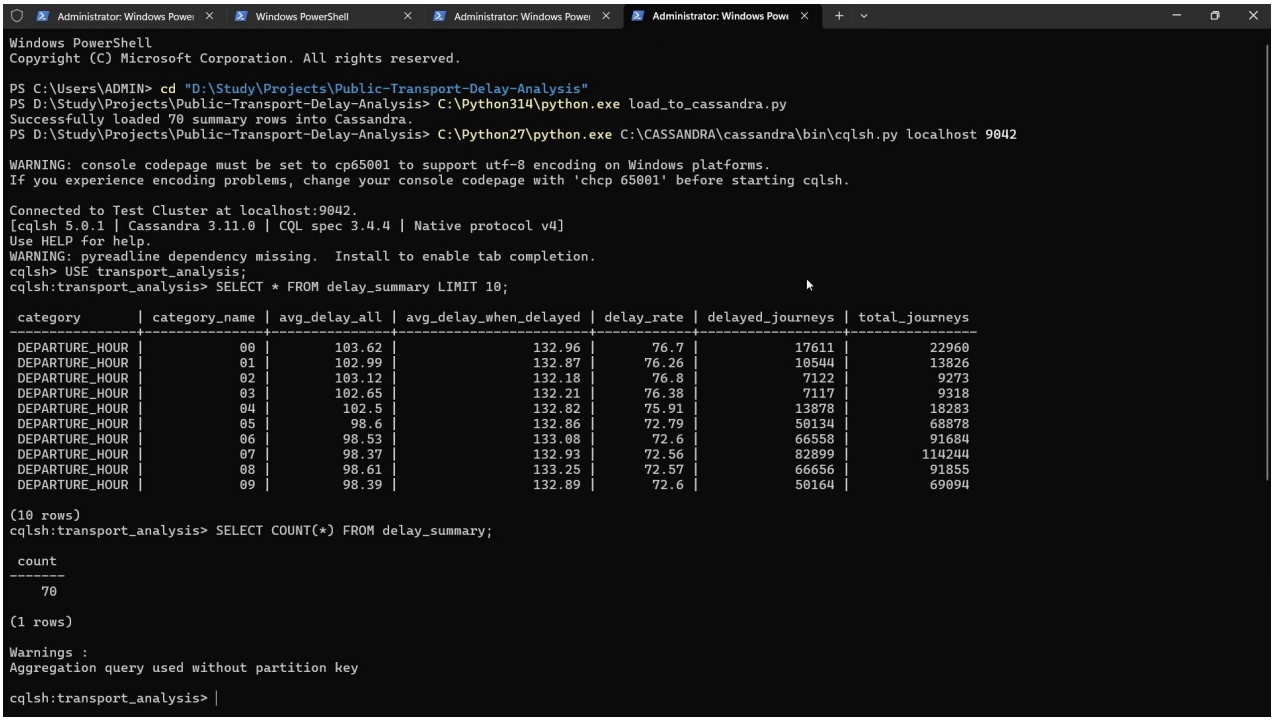


Figure C.3 - Cassandra sample query and SELECT COUNT(*) verification showing 70 summary rows.

The dashboard screenshots in Section 7 are the direct visual evidence for the overview, railway zones, train types, departure hours and delay-cause views. Together, these exhibits document the main storage, processing, persistence and presentation stages of the completed local pipeline.

Appendix D - Reproducibility Checklist

The following sequence summarises the local batch execution used for this project. It assumes the Hadoop and Cassandra services, project files, and Python dependencies are configured.

1. Verify the input in HDFS with: hdfs dfs -ls /public_transport_delay/input/ir_train.csv.
2. Run the batch job with: hadoop jar transport-delay-analysis.jar TransportDelayAnalysis /public_transport_delay/input/ir_train.csv /public_transport_delay/mapreduce_output. The output directory must not already exist.
3. Load the reducer output with: python load_to_cassandra.py. In cqlsh, run SELECT COUNT(*) FROM delay_summary; and verify the recorded count of 70 summary rows.
4. Start the dashboard with: python -m streamlit run app.py. Check the overview and the Railway Zones, Train Types, Departure Hours, and Delay Causes sections.

Completion evidence: the HDFS input path, the MapReduce _SUCCESS marker and part-r-00000 output, the Cassandra row-count query, and the four dashboard sections document the main pipeline stages.